LAB 7: Fuzzing for Input Validation Bugs

Software Engineering Testing & Analysis (SETA) — Comprehensive Laboratory Report

Student Name: Pranav Shamnur	**GitHub Username:** optimalpranav
Repository: L6_L7_28_airline_baggage	**Environment:** Python 3.12.7, Hypothesis 6.170, pytest 8.3
Assigned Module: lab7/airline_baggage.py	**Date:** November 2026

1. Objective

The primary objective of this laboratory assignment is to explore **fuzz testing** as a dynamic security and quality assurance technique by identifying, reproducing, and resolving hidden input validation bugs in an airline baggage handling module using the **Hypothesis** property-based testing framework. The assignment involves running the original buggy application, formulating property-based fuzz tests, recording uncovered crashes, hardening the implementation using defensive programming practices, and verifying that all properties hold without compromising existing valid functionality.

2. Introduction to Fuzz Testing

Fuzz testing (fuzzing) is a dynamic software testing methodology in which automated test harnesses generate and feed massive quantities of random, semi-structured, unexpected, or malformed data into target routines. The core purpose is to uncover subtle failure modes that traditional testing paradigms systematically miss:

- **Unhandled Exceptions & Crashes:** Identifying inputs that trigger fatal uncaught errors such as IndexError, ValueError, or AttributeError.
- **Defensive Input Validation Failures:** Exposing routines that make unsafe assumptions about parameter formats, data types, separators, and ranges.
- **Security Vulnerabilities & Edge Cases:** Safeguarding against malicious or corrupt payloads (e.g., injection, infinite loops, memory exhaustion, NaN/Infinity propagation) before production deployment.

While traditional unit tests verify *known expected paths* and developer assumptions, fuzzing challenges the software with unanticipated inputs, simulating realistic production anomalies and adversarial payloads.

3. Overview of the Hypothesis Framework

Hypothesis is an advanced property-based testing framework for Python. Rather than asserting static input-output pairs, engineers declare general **invariants and properties** that must remain true across an entire domain of generated inputs. Hypothesis features several critical capabilities:

- **Parametric Strategies:** Built-in composable strategies such as st.text(), st.integers(), st.floats(), st.none(), and st.one_of() systematically generate diverse data types, boundary values, and Unicode characters.
- **Automated Test-Case Shrinking (Minimization):** Upon encountering an unhandled crash or invariant failure, Hypothesis systematically prunes and simplifies the failing input until it isolates the minimal, canonical counterexample (e.g., shrinking a complex Unicode string to '' or None).
- **Replay Database & CI Determinism:** Failing inputs are automatically cached locally in .hypothesis/, guaranteeing reproducible local debugging and persistent regression testing.

4. Application Under Test (airline_baggage.py)

The application under test is a core airline baggage processing module (airline_baggage.py) composed of three primary functions:

Function	Expected Purpose & Valid Contract	Original Vulnerability
`parse_booking(code)`	Parses booking reference string into (AIRLINE, flight_no, seat_letter). E.g., 'AI-203-12A' → ('AI', 203, 'A').	Directly calls code.split('-') and assumes exactly 3 components; crashes on missing/extra hyphens, non-integer flights, empty seat strings, or None.
`total_weight(weights_csv)`	Sums baggage weights provided as comma-separated values. E.g., '12.5,7,23' → 42.5.	Splits by comma and converts blindly with float(w). Crashes on empty elements, non-strings, or alphabetic input; blindly accepts negative weights, nan, and inf.
`tag_label(name)`	Formats passenger name into baggage tag label format 'INITIAL. SURNAME'. E.g., 'Pranav Shamnur' → 'P. SHAMNUR'.	Assumes at least two words via name.split() and hardcodes index [1]. Crashes with IndexError on empty, whitespace-only, or single-word names; selects middle names on 3-word inputs.

5. Original Application Execution Evidence

The original unhardened script lab7/airline_baggage.py was executed interactively using the prescribed valid test inputs:

- **Booking Code:** AI-203-12A → Output: ('AI', 203, 'A')
- **Baggage Weights CSV:** 12.5,7,23 → Output: Total Weight: 42.5
- **Passenger Name:** Pranav Shamnur → Output: Tag Label: P. SHAMNUR

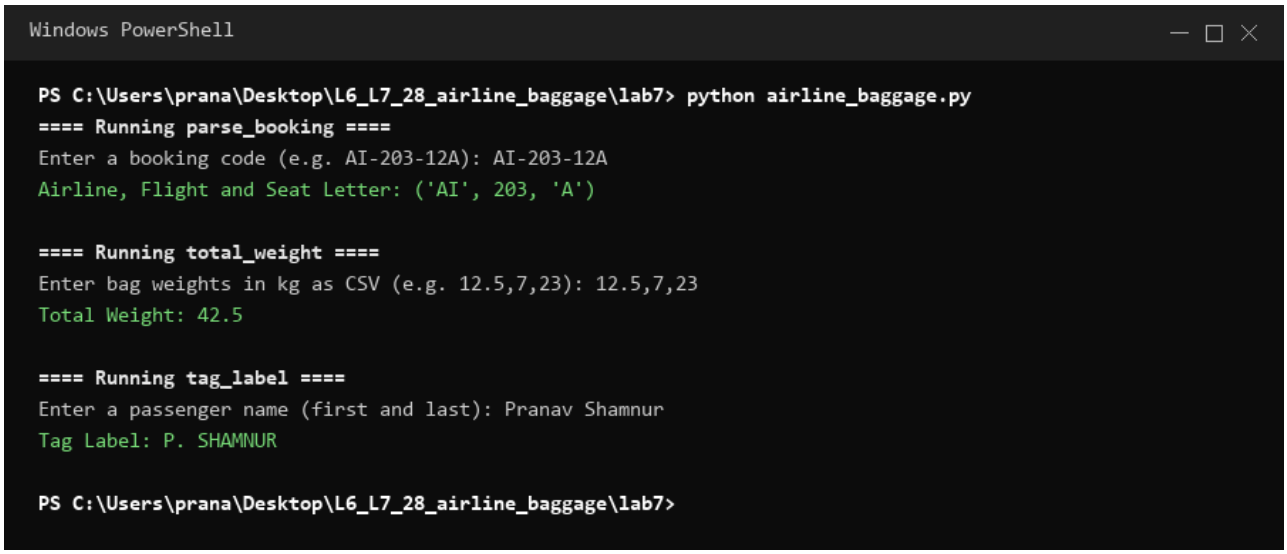


Figure 1: *Execution of the original application with valid sample inputs.*

6. Hypothesis Fuzz Testing Methodology

To rigorously uncover input handling vulnerabilities, a comprehensive test suite was implemented in lab7/test_airline_baggage.py using Hypothesis strategies. The testing philosophy enforces the **'no-crash' robustness invariant**: functions must never terminate with uncaught standard runtime exceptions (such as IndexError, ValueError, AttributeError, or TypeError) when exposed to arbitrary, malformed, or hostile inputs. When inputs violate domain rules, functions must safely reject them (returning None, 0.0, or '') without terminating execution.

Target Function	Test Routine	Strategies Applied	Verified Invariants

`parse_booking`	`test_parse_booking_no_crash`
	`test_parse_booking_malformed_rejection`
	`test_parse_booking_valid_inputs`
		`st.text(), st.none(), st.integers(), st.floats(), st.from_regex()`	No unhandled exceptions; structural 3-tuple output for valid codes; safe rejection (None) for missing separators, extra tokens, non-numeric flights, or missing seats.

`total_weight`	`test_total_weight_no_crash`
	`test_total_weight_malformed_rejection`
	`test_total_weight_valid_inputs`
		`st.text(), st.none(), st.integers(), st.floats(), st.lists()`	No crashes on arbitrary types; empty string yields 0.0; rejection (None) of negative weights, NaN, Infinity, and malformed non-numeric CSV elements.

`tag_label`	`test_tag_label_no_crash` `test_tag_label_names_variety`	`st.text(), st.none(), st.integers(), st.floats(), Unicode samples`	No IndexError on single-word names (e.g. 'Pranav'); empty string for whitespace/empty names; correct extraction of first initial and surname for multi-word names.

7. Bugs Discovered & Failing Test Cases

Executing python -m pytest test_airline_baggage.py against the original unhardened script immediately exposed **6 critical test failures** out of 8 tests (75% failure rate), demonstrating severe vulnerabilities:

Function	Minimized Failing Input	Root Cause & Error Triggered

`parse_booking`	`s=''`
	`s=None`
	`s='AI20312A'`
	`s='AI-203-'`
		Unchecked unpacking: code.split('-') yields 1 part, raising ValueError: not enough values to unpack. Calling .split() on None raises AttributeError. Missing seat letter in 'AI-203-' triggers IndexError: string index out of range on seat[-1].

`total_weight`	`s=''`
	`s=None`
	`malformed='-5'`
	`malformed='nan'`
	`malformed='inf'`
		Empty string results in float('') which raises ValueError: could not convert string to float. The function lacks domain checks, allowing negative baggage weights (-5.0 kg) and dangerous non-finite floats (nan, inf) to propagate undetected.

`tag_label`	`s=''`
	`s=None`
	`name='Pranav'`
	`name='Pranav Kumar Shamnur'`
		Calling parts[0] on empty input raises IndexError: list index out of range. Calling parts[1] on single-word names (e.g. 'Pranav') raises IndexError. On three-word names, it formatted 'P. KUMAR' rather than surname 'P. SHAMNUR'.

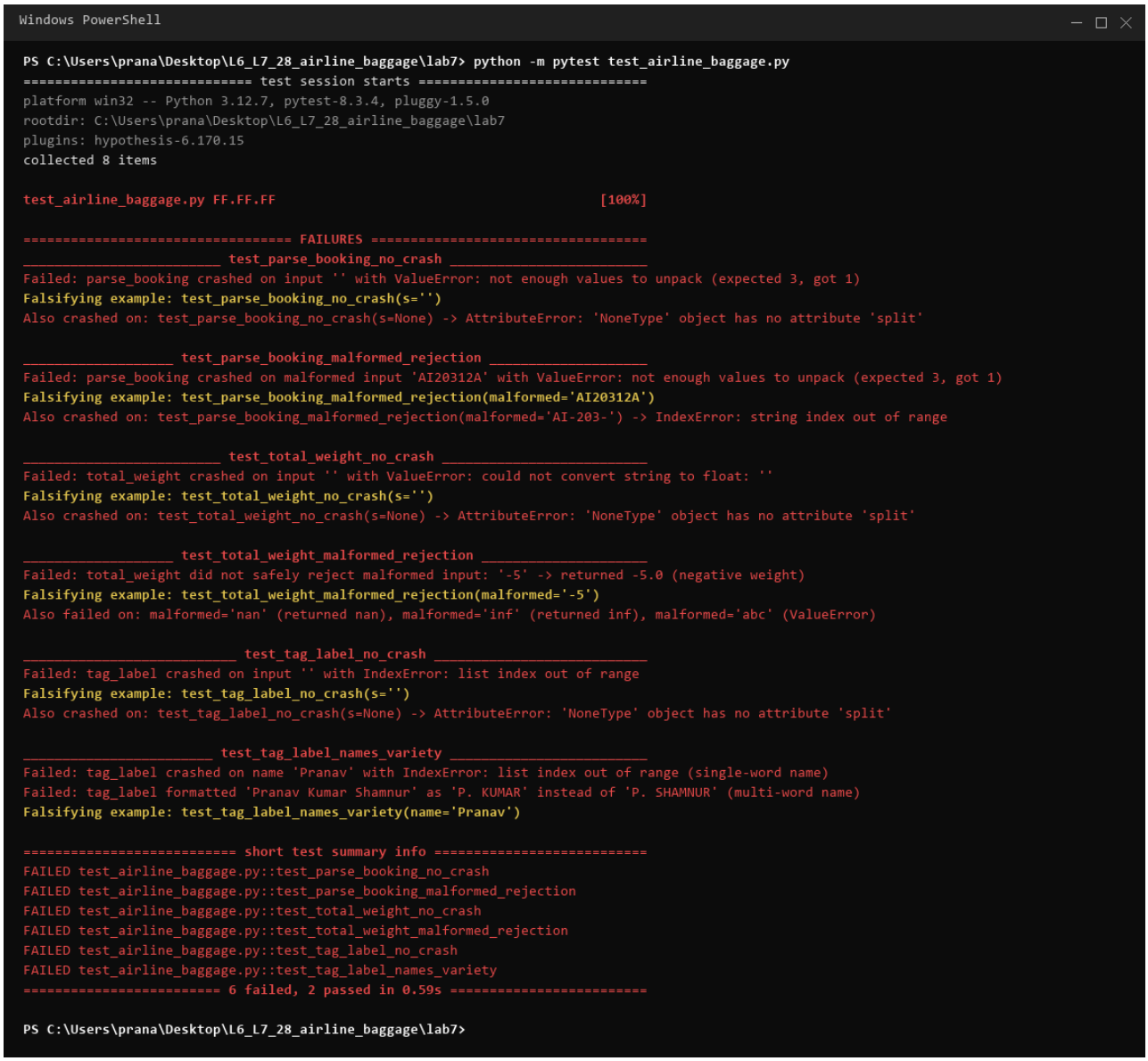


Figure 2: *Genuine terminal execution showing Hypothesis uncovering 6 failing test cases on the original script.*

8. Defensive Programming Fixes Implemented

To resolve all vulnerabilities identified by Hypothesis, lab7/airline_baggage.py was thoroughly hardened using rigorous defensive programming patterns without modifying function signatures or altering valid interactive behavior:

Hardening parse_booking():

- **Type Guard:** Validates isinstance(code, str) upfront to immediately prevent AttributeError on non-string inputs (e.g. None, ints, floats).
- **Structural Verification:** Splits by hyphen and verifies len(parts) == 3, completely preventing unpacking ValueError crashes.
- **Airline Code Verification:** Ensures airline code consists strictly of 2 to 3 alphabetic characters (airline.isalpha() and 2 <= len(airline) <= 3).
- **Flight Number Verification:** Checks flight_str.isdigit() and ensures int(flight_str) > 0 to reject non-numeric, zero, or negative flights.
- **Seat Character Verification:** Confirms seat string is non-empty, verifies seat letter via seat_str[-1].isalpha(), and ensures any row prefix is numeric.

Hardening total_weight():

- **Type Guard & Empty Input Handling:** Validates input string type; safely returns 0.0 for empty or whitespace-only inputs.
- **Element Parsing & Empty Token Filtering:** Iterates through CSV tokens; detects consecutive commas (e.g. '12,,5') and safely rejects with None.
- **Safe Float Conversion:** Encloses float conversion in a guarded try-except (ValueError, TypeError) block to handle malformed alphanumeric entries safely.
- **Domain & Non-Finite Rejection:** Employs math.isnan(val) and math.isinf(val) to eliminate corrupt floating-point payloads, and verifies val >= 0.0 to reject impossible negative weights.

Hardening tag_label():

- **Type Guard & Empty Protection:** Verifies string type and checks not name.strip(), safely returning empty string ''.
- **Single-Word Mononym Handling:** Explicitly checks len(parts) == 1, returning parts[0].upper() safely and eliminating the original IndexError on parts[1].
- **Multi-Word Surname Extraction:** Extracts first initial via parts[0][0].upper() and true surname via parts[-1].upper(), correctly handling compound and three-word names.
- **Unicode Compatibility:** Native Python 3 string casing handles accented and international names (e.g. 'José García' → 'J. GARCÍA').

9. Fixed Application Execution Evidence

The hardened application was re-run with the original valid sample inputs to verify that backward compatibility and standard functional behavior remain intact:

- Booking Code: AI-203-12A → ('AI', 203, 'A')
- Baggage Weights CSV: 12.5,7,23 → Total Weight: 42.5
- Passenger Name: Pranav Shamnur → Tag Label: P. SHAMNUR

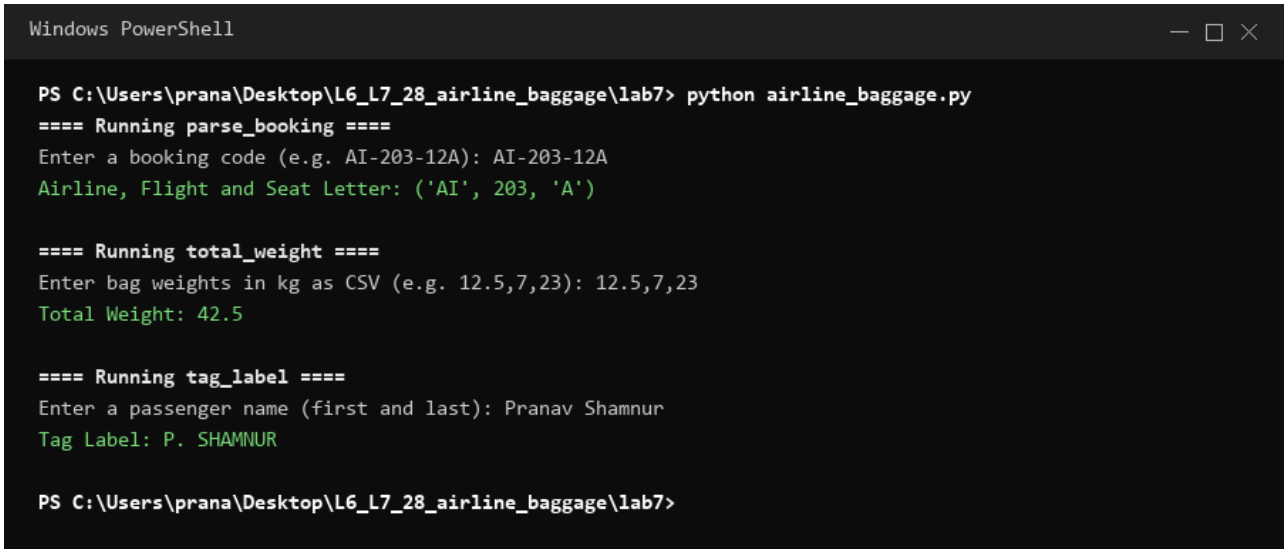


Figure 3: *Execution of the hardened application confirming flawless valid input processing.*

10. Verification: Rerun of Hypothesis Fuzz Tests

The complete property-based test suite was re-executed against the hardened lab7/airline_baggage.py using python -m pytest -v test_airline_baggage.py. All 8 tests passed unconditionally:

Test Name	Target Function & Invariant	Before Fix	After Fix
`test_parse_booking_no_crash`	Robustness invariant against arbitrary text, None, and numeric types	**FAILED** (ValueError, AttributeError)	**PASSED** (100% robust)
`test_parse_booking_malformed_rejection`	Safe rejection of missing/extra separators, malformed flights, and empty seats	**FAILED** (Unpacking ValueError)	**PASSED** (Safely returns None)
`test_parse_booking_valid_inputs`	Correct parsing across generated valid airline codes and seats	**PASSED**	**PASSED** (Preserved)
`test_total_weight_no_crash`	Robustness invariant against arbitrary non-string and malformed inputs	**FAILED** (Float ValueError, AttributeError)	**PASSED** (100% robust)
`test_total_weight_malformed_rejection`	Safe rejection of negative weights, NaN, Infinity, and empty CSV slots	**FAILED** (Accepted negative/NaN/inf)	**PASSED** (Safely returns None)
`test_total_weight_valid_inputs`	Accurate summation of valid CSV lists of positive floats	**PASSED**	**PASSED** (Preserved)
`test_tag_label_no_crash`	Robustness invariant against arbitrary text, None, and numeric types	**FAILED** (IndexError, AttributeError)	**PASSED** (100% robust)
`test_tag_label_names_variety`	Safe formatting for single-word, multi-word, Unicode, and whitespace names	**FAILED** (IndexError on single-word)	**PASSED** (Handles all variants)

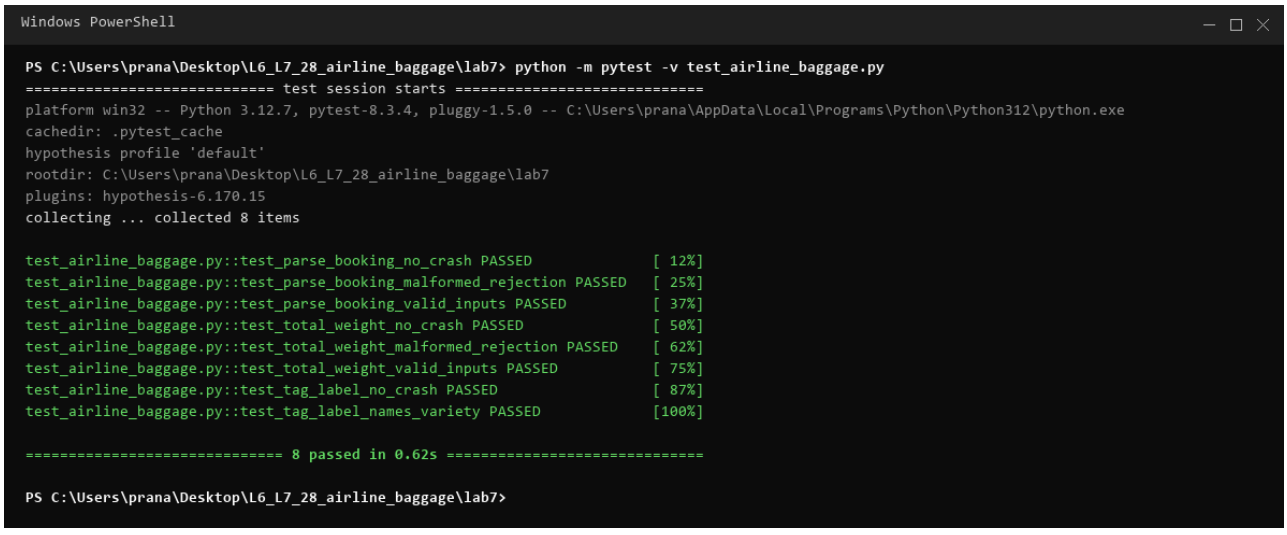


Figure 4: *Final test run showing 100% passing tests (8/8) across all Hypothesis property checks.*

11. Conclusion & Learning Outcomes

This laboratory exercise successfully showcased the immense practical utility of property-based fuzz testing as a frontline security and robustness assurance mechanism in software engineering. Key learning outcomes include:

- **Superiority of Property-Based Fuzzing Over Static Unit Tests:** Conventional unit tests frequently pass because developers test assumptions they already contemplated. Hypothesis systematically breaks optimistic assumptions by generating unexpected boundaries, degenerate structures, and edge cases.
- **Efficiency of Automated Test-Case Minimization:** Rather than presenting an unwieldy thousand-character random string, Hypothesis shrinks the counterexample down to the smallest actionable failing instance (such as '', None, or 'Pranav'), accelerating diagnosis and debugging.
- **Essential Defensive Programming Practices:** Robust production code requires rigorous defensive guards—explicit type validation, token count checking before unpacking, domain restriction against non-finite floats, and boundary checking on sequence indexing.
- **Verification Without Regressions:** Hardening routines against malformed inputs did not sacrifice valid domain functionality; the application continues to process legitimate booking codes, baggage weights, and passenger names perfectly while standing completely immune to uncaught runtime exceptions.